

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук

Образовательная программа «Дизайн и разработка информационных продуктов»

СОГЛАСОВАНО

Научный руководитель
Приглашённый преподаватель:
Факультет компьютерных наук /
Департамент больших данных и
информационного поиска / Базовая
кафедра Т-Банка

_____ А. А. Шкель
«__» _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель
образовательной программы
«Дизайн и разработка
информационных продуктов»,
преподаватель базовой кафедры Т-
Банка

_____ Д. Д. Син
«__» _____ 2026 г.

ПЛАТФОРМА ДЛЯ NO-CODE АВТОМАТИЗАЦИИ БИЗНЕС-ПРОЦЕССОВ.

Руководство оператора (индивидуальная часть)

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.09.03-04 34 03-1-ЛУ

Исполнители:

Студент группы БДРИП242

_____ / А. А. Клушин /

«__» _____ 2026 г.

Москва 2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл.	

УТВЕРЖДЕН

RU.17701729.09.03-04 34 03-1-ЛЮ

ПЛАТФОРМА ДЛЯ NO-CODE АВТОМАТИЗАЦИИ БИЗНЕС-ПРОЦЕССОВ.

Руководство оператора (индивидуальная часть)

RU.17701729.09.03-04 34 03-1

Листов 18

Москва 2026

Инов.№ подп	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

АННОТАЦИЯ

Настоящий документ является индивидуальной частью руководства оператора для серверной части программы «Платформа для no-code автоматизации бизнес-процессов». Документ содержит инструкции по развёртыванию, конфигурации и эксплуатации серверной части, включая REST API, песочницу Code Node, триггеры, работу с MinIO и Swagger UI.

Настоящий документ разработан в соответствии с ГОСТ 19.505-79.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 34 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. НАЗНАЧЕНИЕ ПРОГРАММЫ	4
1.1. Функциональное назначение	4
1.2. Эксплуатационное назначение	4
2. УСЛОВИЯ ВЫПОЛНЕНИЯ ПРОГРАММЫ	5
2.1. Минимальный состав технических средств	5
2.2. Минимальный состав программных средств	5
2.3. Требования к оператору	5
3. ВЫПОЛНЕНИЕ ПРОГРАММЫ	6
3.1. Развёртывание через Docker Compose (рекомендованный способ)	6
3.2. Сборка из исходников	6
3.3. Запуск JAR вручную	7
3.4. Работа со Swagger UI	8
3.5. Управление workflow через REST API	8
3.6. Запуск workflow и работа с результатами	9
3.7. Управление триггерами	9
3.8. Управление версиями	10
3.9. MinIO Console	10
3.10. Миграции схемы Liquibase	11
3.11. Запуск тестов и анализ покрытия	11
3.12. Логи и отладка	11
3.13. Подключение к WebSocket для тестирования	12
4. СООБЩЕНИЯ ОПЕРАТОРУ	13
5. ИНТЕГРАЦИЯ С ВНЕШНИМИ СИСТЕМАМИ	15
6. ВОССТАНОВЛЕНИЕ ПОСЛЕ СБОЕВ	16
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	17

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 34 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. НАЗНАЧЕНИЕ ПРОГРАММЫ

1.1. Функциональное назначение

Серверная часть программы «Платформа для no-code автоматизации бизнес-процессов» представляет собой модульный монолит на Kotlin 2.2 / Spring Boot 3.5.3 (JDK 24), обеспечивающий: REST API управления workflow с автоматической документацией OpenAPI 3.0; движок параллельного исполнения графов workflow по алгоритму Кана + CompletableFuture в пуле потоков; изолированную Docker-песочницу для пользовательского Python- и JavaScript-кода; интеграцию с реляционной базой PostgreSQL 16 через JDBC и миграции Liquibase; контент-адресуемое хранилище MinIO для конфигов узлов; WebSocket-канал STOMP поверх SockJS для трансляции событий исполнения в реальном времени; систему триггеров трёх типов (manual, scheduler с cron-выражениями, webhook со сгенерированным токеном).

1.2. Эксплуатационное назначение

Серверная часть предназначена для развёртывания на Linux-сервере (Ubuntu 22.04 LTS и совместимые) или в составе оркестрированной среды контейнеров (Docker Compose, Kubernetes). Обслуживает HTTP-запросы клиентской части (Angular SPA) по протоколам REST и WebSocket на порту 8080. Целевая аудитория настоящего руководства — системные администраторы, DevOps-инженеры и разработчики, разворачивающие и эксплуатирующие серверную часть, а также интегрирующие её с внешними системами через REST API.

Промышленная (development и demonstration) инстанция серверной части развёрнута и публично доступна по адресу <https://fluxpilot.ru/> интерактивная документация REST API — по адресу <https://fluxpilot.ru/v1/swagger-ui.html>. Все примеры команд curl в настоящем документе, использующие <http://localhost:8080>, могут быть выполнены и против промышленной инстанции с заменой базового адреса на <https://fluxpilot.ru> (с учётом TLS).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 34 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. УСЛОВИЯ ВЫПОЛНЕНИЯ ПРОГРАММЫ

2.1. Минимальный состав технических средств

Процессор не менее 4 ядер архитектуры x86_64; оперативная память не менее 8 ГБ (рекомендуется 16 ГБ для одновременного запуска backend, базы данных, MinIO, Docker-контейнеров песочницы и инструментов мониторинга); дисковое пространство не менее 20 ГБ свободного на корневом разделе (расходуется на образы Docker, данные PostgreSQL, объекты MinIO, логи приложения); стабильное подключение к сети Интернет (для первоначальной загрузки Docker-образа python:3.11-slim, используемого песочницей Code Node, а также для исходящих HTTP-запросов узлов workflow к внешним сервисам).

2.2. Минимальный состав программных средств

Операционная система: Linux (Ubuntu 22.04 LTS и совместимые), либо macOS 12+, либо Windows 10/11 с установленной подсистемой WSL2. JDK 24 для запуска fat-JAR при ручной сборке; Apache Maven 3.9+ для пересборки из исходников; Docker Engine 20.10+ с активным docker daemon (используется как для запуска вспомогательной инфраструктуры через Docker Compose, так и для исполнения Code Node-узлов); PostgreSQL 16 (поднимается через Docker Compose); MinIO (поднимается через Docker Compose). Утилиты, рекомендуемые для эксплуатации: curl для ручных REST-запросов, psql для прямых SQL-запросов к базе, любой современный браузер для работы со Swagger UI и MinIO Console.

2.3. Требования к оператору

Оператор должен обладать базовыми навыками работы с командной строкой Linux/macOS, понимать концепции HTTP, REST, Docker-контейнеризации. Для решения нестандартных задач (например, расследование инцидентов, ручное вмешательство в состояние БД) рекомендуется владение SQL и знание основ Spring Boot. Для использования REST API напрямую (минуя клиентскую часть Angular) — владение JSON, curl/Postman и понимание спецификации OpenAPI 3.0.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 34 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. ВЫПОЛНЕНИЕ ПРОГРАММЫ

3.1. Развёртывание через Docker Compose (рекомендованный способ)

Полное развёртывание development-окружения выполняется одной командой.

Шаг 1. Клонировать репозиторий проекта:

```
git clone https://github.com/arklual/cursach.git && cd cursach
```

Шаг 2. Запустить вспомогательную инфраструктуру (PostgreSQL и MinIO):

```
docker compose -f deploy/docker-compose.dev.yml up -d
```

Команда поднимает два контейнера: PostgreSQL 16 на порту 5433 (внутри контейнера — стандартный 5432) и MinIO на портах 6005 (API) и 6006 (веб-консоль). Контейнеры именованы flux-postgres-dev и flux-minio-dev, имеют ограничения по памяти и используют именованные volumes для персистентности данных.

Шаг 3. Дождаться готовности сервисов (проверка через healthcheck):

```
docker compose -f deploy/docker-compose.dev.yml ps
```

Состояние «healthy» обоих контейнеров достигается обычно за 10–20 секунд.

Шаг 4. Запустить серверную часть из директории backend/:

```
cd backend && mvn spring-boot:run
```

Spring Boot стартует на порту 8080. Готовность фиксируется в логе строкой «Started AllaApplication in N seconds (process running for N)». Применение миграций Liquibase происходит автоматически на старте, прогресс отражается в логах с префиксом liquibase:.

Альтернативный способ — продакшен-окружение. Для полного развёртывания production-окружения с собранными Docker-образами backend и frontend, Nginx с TLS-терминированием:

```
docker compose -f deploy/docker-compose.prod.yml up -d
```

Поднимает PostgreSQL (без проброса портов наружу), MinIO, docker-socket-проху (для безопасного доступа backend к docker daemon хоста), сам backend, frontend на Nginx с SSL на портах 80/443. Здесь используется единый docker-network, доступ извне — только через Nginx.

3.2. Сборка из исходников

Сборка backend в исполняемый fat-JAR:

```
cd backend && mvn clean verify
```

Результат: backend/target/aelevena-*.jar (обычно 50–80 МБ). В процессе сборки автомати-

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 34 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

чески прогоняются модульные и интеграционные тесты (Testcontainers поднимает временные PostgreSQL и MinIO для интеграционных тестов), формируется HTML-отчёт покрытия в backend/target/site/jacoco/. При неуспехе хотя бы одного теста сборка завершается с ненулевым кодом выхода.

Сборка без тестов (для быстрых пересборок в development):

```
cd backend && mvn package -DskipTests
```

3.3. Запуск JAR вручную

После сборки fat-JAR можно запустить напрямую через java:

```
java -jar backend/target/aelevena-*.jar
```

Опционально с переопределением профиля Spring:

```
java -Dspring.profiles.active=production -jar backend/target/aelevena-*.jar
```

Конфигурация осуществляется через переменные окружения (имеют приоритет над application.properties):

Переменная	По умолчанию	Назначение
SPRING_DATASOURCE_URL	jdbc:postgresql://localhost:5432/a11a	JDBC URL PostgreSQL
SPRING_DATASOURCE_USERNAME	flux	Пользователь PostgreSQL
SPRING_DATASOURCE_PASSWORD	flux	Пароль PostgreSQL
APP_S3_ENDPOINT	http://localhost:9000	Адрес MinIO API
APP_S3_ACCESS_KEY	flux	Access key MinIO
APP_S3_SECRET_KEY	flux-secret	Secret key MinIO
APP_S3_BUCKET	a11a-blobs	Имя bucket для blob-ов
APP_BLOB_MAX_SIZE_BYTES	104857600	Максимальный размер одного blob (100 МБ)
APP_EXECUTOR_POOL_SIZE	16	Размер пула параллельного исполнения
WS_ALLOWED_ORIGINS	http://localhost:4200	Разрешённые origin для WebSocket (CORS)
APP_SEED_DEMO_WORKFLOWS	true	Посевать ли демонстрационные workflow на старте
SERVER_PORT	8080	Порт HTTP-сервера

Таблица 1. Основные переменные окружения серверной части

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 34 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3.4. Работа со Swagger UI

После запуска backend интерактивная документация REST API доступна по адресу:

`http://localhost:8080/v1/swagger-ui.html`

Интерфейс позволяет: просматривать все эндпоинты, сгруппированные по тегам (Workflows, Versions, Runs, Triggers, Webhooks); видеть схемы запросов и ответов; выполнять пробные запросы кнопкой «Try it out» с автоматической подстановкой токенов/идентификаторов; импортировать спецификацию OpenAPI 3.0 в Postman/Insomnia через URL `http://localhost:8080/v1/v3/api-docs`.

3.5. Управление workflow через REST API

Основные сценарии работы с workflow напрямую через REST.

Создание workflow. Базовый запрос без графа:

```
curl -X POST http://localhost:8080/v1/workflows \
  -H "Content-Type: application/json" \
  -d '{"name":"My first workflow","description":"Test workflow"}'
```

В ответ возвращается JSON с полем `id` (UUID), идентификатором созданного workflow. После создания workflow содержит пустой граф; для добавления узлов и связей необходимо использовать клиентскую часть Angular или прямой PUT к версии графа.

Получение списка workflow:

```
curl http://localhost:8080/v1/workflows
```

Возвращает массив объектов с полями `id`, `name`, `description`, `currentVersionId`, `createdAt`.

Получение workflow с актуальной ревизией:

```
curl http://localhost:8080/v1/workflows/{id}
```

Возвращает полный объект workflow с полем `graphSkeleton`, содержащим текущую структуру графа.

Обновление workflow (создание новой ревизии):

```
curl -X PUT http://localhost:8080/v1/workflows/{id} \
  -H "Content-Type: application/json" \
  -d '{"name":"...", "description":"...", "graphSkeleton":{...}}'
```

Каждый успешный PUT создаёт новую запись в `workflow_revision` с инкрементальным `revision_number`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 34 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Удаление workflow: `curl -X DELETE http://localhost:8080/v1/workflows/{id}` — каскадно удаляет все ревизии, версии, запуски и узлы запусков.

3.6. Запуск workflow и работа с результатами

Запуск workflow (ручной триггер):

```
curl -X POST http://localhost:8080/v1/workflows/{id}/runs \
  -H "Content-Type: application/json" \
  -d '{"input":{"hello":"world"}}'
```

Тело запроса опционально; если указано, передаётся как input первому узлу. Ответ: 202 Accepted с идентификатором запуска workflowRunId.

Получение состояния запуска:

```
curl http://localhost:8080/v1/workflow-runs/{runId}
```

Возвращает текущий статус (pending, running, success, failed), время начала/окончания, список node_run.

Получение детального результата конкретного узла:

```
curl http://localhost:8080/v1/node-runs/{nodeRunId}
```

Возвращает inputJson, outputJson, errorMessage (если applicable), configHash.

История запусков workflow:

```
curl http://localhost:8080/v1/workflows/{id}/runs
```

Возвращает массив запусков, отсортированный по started_at DESC.

Запуск одной ноды для отладки. Серверная часть предоставляет REST-эндпойнт запуска одиночной ноды workflow с произвольным входным JSON-телом, без полного прогона графа. Эндпойнт принимает идентификатор целевой ноды и тело входных данных, исполняет только этот узел с использованием его текущей конфигурации и возвращает результат исполнения (выходные данные либо диагностическое сообщение об ошибке) в синхронном режиме; параллельно создаётся запись запуска с пометкой отладочного исполнения, что обеспечивает попадание результата в обычную историю запусков workflow. Указанная функциональность предназначена для отладки конфигурации отдельных узлов (выражения фильтрации, пользовательского Python- и JavaScript-кода, шаблонов HTTP-запросов) и для разработчиков, интегрирующих платформу с внешними системами.

3.7. Управление триггерами

Создание scheduler-триггера:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 34 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```
curl -X POST http://localhost:8080/v1/workflows/{id}/triggers \
  -H "Content-Type: application/json" \
  -d '{"type":"scheduler","config":{"cron":"0 0 9 * * *"},"enabled":true}'
```

Cron-выражение в формате Spring (6 полей: секунды, минуты, часы, день месяца, месяц, день недели). Триггер регистрируется в TriggerScheduler сразу после создания (горячая перезагрузка расписания).

Создание webhook-триггера:

```
curl -X POST http://localhost:8080/v1/workflows/{id}/triggers \
  -H "Content-Type: application/json" \
  -d '{"type":"webhook","enabled":true}'
```

В ответе возвращается поле token (UUID). Webhook-URL формируется как `http://localhost:8080/v1/webhook/{token}`. Этот URL необходимо настроить во внешней системе как принимающий URL для нужного события.

Включение/выключение триггера:

```
curl -X PATCH http://localhost:8080/v1/workflows/{id}/triggers/{tid} \
  -H "Content-Type: application/json" \
  -d '{"enabled":false}'
```

3.8. Управление версиями

Список ревизий и версий workflow:

```
curl http://localhost:8080/v1/workflows/{id}/versions
```

Возвращает массив объектов `workflow_version` с полями `id`, `versionTag`, `rootRevisionId`, `createdAt`.

Создание именованного snapshot:

```
curl -X POST http://localhost:8080/v1/workflows/{id}/versions \
  -H "Content-Type: application/json" \
  -d '{"versionTag":"v1.0-stable"}'
```

Создаёт версию, ссылающуюся на текущую (последнюю) ревизию workflow.

3.9. MinIO Console

Веб-консоль MinIO доступна по адресу `http://localhost:6006` (development) или соответствующему адресу production. Логин/пароль — `flux / flux-secret` (development) или из конфигурации production. Консоль позволяет визуально просматривать содержимое bucket `alla-`

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 34 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

blobs, проверять content-addressed-объекты (имена объектов = SHA-256-хеши их содержимого), скачивать отдельные файлы для отладки.

3.10. Миграции схемы Liquibase

Миграции применяются автоматически при запуске backend. Файлы changeset расположены в backend/src/main/resources/db/changelog/changes/. Реализовано пять инкрементальных changeset-ов: 001-init.yaml, 002-trigger-node-id.yaml, 003-restore-after-drop.yaml, 004-add-is-demo.yaml, 005-workflow-snapshots.yaml (см. индивидуальную ПЗ, раздел 3.2.10). При необходимости ручного применения миграций (например, перед прогоном тестов на отдельной БД):

```
cd backend && mvn liquibase:update -Dliquibase.url=jdbc:postgresql://localhost:5432/test-db
```

Состояние миграций отслеживается в таблицах databasechangelog (история applied changeset) и databasechangeloglock (блокировка от одновременных applies).

3.11. Запуск тестов и анализ покрытия

Полный прогон автоматических тестов: `cd backend && mvn verify`. Результаты — в backend/target/surefire-reports/ (JUnit XML по каждому тестовому методу) и backend/target/site/jacoco/index.html (HTML-отчёт покрытия). Для CI-режима с автоматической остановкой при сбое: `mvn verify -ff (--fail-fast)`.

Запуск отдельного тестового класса:

```
mvn test -Dtest=DataflowNodeExecutorsTest
```

Запуск одного метода:

```
mvn test -Dtest=DataflowNodeExecutorsTest#filterRetainsMatchingItems
```

3.12. Логи и отладка

Логи backend выводятся в stdout (стандартный поток), для production-окружения рекомендуется направлять их в журналируемую систему (journald, syslog) или агрегатор логов (ELK, Loki). Уровень логирования управляется через переменные окружения вида `LOGGING_LEVEL_RU_STARTEM_AELEVENA=DEBUG` или через файл `application.properties` (`logging.level.ru.startem.aelevena=DEBUG`). При сбоях исполнения workflow дополнительные диагностические данные сохраняются в таблице `node_run.error_message`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 34 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3.13. Подключение к WebSocket для тестирования

Для проверки WebSocket-канала со стороны эксплуатации можно использовать STOMP-клиент. Пример с stompit-cli:

```
npm install -g stompit-cli
```

```
stompit-cli ws://localhost:8080/v1/ws
```

```
> CONNECT login: guest passcode: guest
```

```
> SUBSCRIBE id: sub-0 destination: /topic/workflows/{workflowId}/graph
```

После подписки запуски выбранного workflow будут транслировать события в этот канал.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 34 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4. СООБЩЕНИЯ ОПЕРАТОРУ

В процессе эксплуатации серверной части оператор может встретить следующие сообщения в логах, REST-ответах или WebSocket-событиях:

Сообщение	Описание	Рекомендуемое действие
Node error: <details>	Сбой исполнения узла workflow	Открыть инспектор узла, проверить конфигурацию; сверить с error_message
Connection timeout: <url>	Превышен тайм-аут HTTP-запроса к внешнему URL	Проверить доступность URL, увеличить тайм-аут в конфиге узла
Sandbox timeout	Код в Code Node выполнялся дольше тайм-аута	Оптимизировать код, увеличить тайм-аут в конфиге узла (максимум 300 с)
Sandbox OOM	Code Node превысил лимит памяти 256 МБ	Сократить объёмы данных в коде, разделить обработку
Sandbox network denied	Попытка сетевого обращения в Code Node	Использовать отдельный HTTP-узел перед Code Node
Sandbox write denied	Попытка записи на FS в Code Node	Использовать только in-memory переменные
Cycle detected in workflow graph	Обнаружен цикл в графе workflow	Открыть редактор, удалить циклическую связь
Dependency failed: <nodeId>	Узел-предок завершился с ошибкой	Исправить ошибку в узле-предке, перезапустить
Validation error: <field>	Некорректный JSON или нарушены ограничения	Проверить структуру тела запроса, сверить со схемой Swagger UI
Cron syntax error	Невалидное cron-выражение в scheduler-триггере	Проверить формат — 6 полей Spring (секунды минуты часы день месяц день_недели), использовать crontab.guru в режиме Spring-cron

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 34 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Сообщение	Описание	Рекомендуемое действие
Webhook token not found	Запрос на webhook-URL с несуществующим токеном	Проверить токен, при необходимости пересоздать триггер
Liquibase changelog lock	Блокировка миграций (другой инстанс применяет)	Дождаться завершения или вручную очистить databaseloglock (только при уверенности)
PostgreSQL connection refused	Backend не может подключиться к БД	Проверить, что PostgreSQL запущен, доступен по адресу из SPRING_DATASOURCE_URL
MinIO bucket not found	Не существует bucket для blob-ов	Создать вручную через MinIO Console или установить APP_S3_BUCKET_AUTO_CREATE=true
Docker daemon not available	Code Node не может запустить контейнер	Проверить, что docker daemon запущен и пользователь backend имеет права на /var/run/docker.sock

Таблица 2. Сообщения оператору и реакция

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 34 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

5. ИНТЕГРАЦИЯ С ВНЕШНИМИ СИСТЕМАМИ

Серверная часть может интегрироваться с внешними системами по нескольким сценариям.

Webhook-источник. Внешняя система (например, платёжный шлюз, мессенджер, форма обратной связи) настраивается на отправку HTTP POST-запроса с произвольным JSON-телом на сгенерированный webhook-URL. Полученное тело становится входом первого узла workflow.

HTTP-узлы. В рамках workflow можно отправлять запросы к произвольным внешним API (REST, SOAP, GraphQL) с настраиваемыми методами, заголовками и телами. Аутентификация через заголовок Authorization, токены пробрасываются как обычные строковые значения.

Code Node. Произвольная программа на Python или JavaScript для нестандартной обработки. Изолирована в Docker-контейнере без сети, поэтому для интеграций следует использовать HTTP-узлы перед/после Code Node.

MinIO API. Объектное хранилище MinIO предоставляет полноценный S3-совместимый API; внешние инструменты (резервное копирование, миграция данных) могут работать с ним через стандартные AWS-инструменты (aws s3 ... с переопределением endpoint).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 34 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

6. ВОССТАНОВЛЕНИЕ ПОСЛЕ СБОЕВ

При корректном развёртывании через Docker Compose сбой backend (например, OOM, kernel panic, ручной `docker stop`) не приводит к потере данных: PostgreSQL и MinIO сохраняют состояние на дисковых volume. После перезапуска backend (`docker restart` или повторный `mvn spring-boot:run`) система продолжит работу. Незавершённые `workflow_run` остаются в статусе `running` и не возобновляются автоматически (для предотвращения дублирующего исполнения побочных эффектов через HTTP-узлы и webhook); они могут быть отмечены как `interrupted` через прямой SQL-апдейт после ручного анализа.

При сбое PostgreSQL восстановление выполняется штатными механизмами Docker volume + WAL-логи. При сбое MinIO — аналогично, либо через восстановление из резервной копии (рекомендуется настроить регулярные снимки bucket через `mc mirror`).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 34 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Spring Boot Reference Documentation [Электронный ресурс]. — URL: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/> (дата обращения: 05.12.2025).
2. Docker Engine Documentation [Электронный ресурс]. — URL: <https://docs.docker.com/engine/> (дата обращения: 18.03.2026).
3. PostgreSQL 16 Documentation [Электронный ресурс]. — URL: <https://www.postgresql.org/docs/16/> (дата обращения: 12.12.2025).
4. MinIO Object Storage Documentation [Электронный ресурс]. — URL: <https://min.io/docs/minio/linux/index.html> (дата обращения: 20.02.2026).
5. Liquibase Documentation [Электронный ресурс]. — URL: <https://docs.liquibase.com/> (дата обращения: 15.12.2025).
6. springdoc-openapi Documentation [Электронный ресурс]. — URL: <https://springdoc.org/> (дата обращения: 05.02.2026).
7. STOMP Protocol Specification [Электронный ресурс]. — URL: <https://stomp.github.io/stomp-specification-1.2.html> (дата обращения: 12.03.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 34 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]